

Lab 1: Functions & Control

- Due: Wednesday 09/02 @ 11:59pm
- Points: 1
- Download: lab01.zip

Attendance

You need to submit the lab problems in addition to attending to get credit for lab. Students in mega lab only need to submit the lab problems. Preceptor interviews are highly recommended but not required for lab submissions.

If you are in regular lab, your TA will come around during lab to check you in. If you didn't attend for a good reason (such as being sick), fill out this form (within 2 weeks of your lab): attendance form.

PrairieLearn and PrairieTest

This semester we will be using PrairieLearn and PrairieTest for our quizzes.

PrairieLearn is where you will access any practice quizzes as well as view your quiz scores. PrairieTest will be used to sign up for a quiz slot (for mega lab students) and to take the quiz (for all students).

Please visit each website and accept the invitation for this course. If you have not received an invitation, please post on Ed and we will send you the invite.

Required Questions

If you feel stuck, try watching some walkthrough videos.

Review

Here are the most common ways to run Python on a file from the VS Code terminal. (The examples use lab00.py, but these commands work for any assignment.)

Using no command-line options will run the code in the file you provide and return you to the command line. If your file just contains function definitions, you'll see no output unless there is a syntax error.

```
python3 lab00.py
```

-i: The -i option runs the code in the file you provide, then opens an interactive session (with a >>> prompt). You can then evaluate expressions such as calling functions you defined. To exit, type exit(). You can also use the keyboard shortcut Ctrl-D on Linux/Mac machines or Ctrl-Z Enter on Windows.

If you edit the Python file while running it interactively, you will need to exit and restart the interpreter in order for those changes to take effect.

Here's how we can run lab00.py interactively:

```
python3 -i lab00.py
```

If the python3 command doesn't work, please try using python or py. You can switch all of the copy/paste commands on this page from python3 to python or py using the toggle at the top right.

Here are examples of the three division-related operators in Python 3.

True division: / (decimal division)

```
>>> 1 / 5
0.2
>>> 25 / 4
6.25
>>> 4 / 2
2.0
```

Floor division: // (integer division; discards the remainder)

```
>>> 1 // 5
0
>>> 25 // 4
6
>>> 4 // 2
2
```

Modulo: % (the remainder after dividing)

```
>>> 1 % 5
1
>>> 25 % 4
1
>>> 4 % 2
0
```

A `ZeroDivisionError` occurs when dividing by 0 with any of these operators.

The remainder when dividing integer n by integer k is the smallest r such that $k*d + r$ equals n for some integer d .
The remainder when dividing n ...

- by 10 is the last digit of n
- by 100 is the last two digits of n
- by $\text{pow}(10, p)$ is the last p digits of n for positive integer p .

One useful technique involving the % operator is to check whether a number x is divisible by another number y :

```
x % y == 0
```

For example, in order to check if x is an even number: $x \% 2 == 0$

Most functions that you define will contain a return statement that provides the value of the call expression that was used to call the function.

When Python executes a return statement, the function call terminates immediately. If Python reaches the end of the function body without executing a return statement, the function returns `None`.

In contrast, the print function is used to display values. Unlike a return statement, when Python evaluates a call to print, the function containing that print call does not terminate immediately.

```
def what_prints():
    print('Hello World!')
    return 'Goodbye.'
    print('What about me?')

>>> what_prints()
Hello World!
'Goodbye.'
```

Notice also that print displays just the text within the quotes, while the return value quotes are displayed as well.

Only the returned value, not the printed one, is the value of the call expression. Printed values are always displayed right away, but whether or when return values are displayed depends on the rest of the code.

```
>>> result = what_prints()
Hello World!
>>> 2 + 2
4
>>> result
'Goodbye.'
```

What Would Python Display? (WWPD)

Q1: Return and Print

Predict what Python will display by running this unlocking session. Please review the dropdowns above if you get stuck.

```
python3 -m pytest -k return_and_print --unlock

>>> def welcome():
...     print('Go')
...     return 'hello'
>>> def cal():
...     print('Bears')
...     return 'world'
>>> welcome()

_____
_____
>>> print(welcome(), cal())
```


Q2: WWPD: What If?

Predict what Python will display by running this unlocking session:

Hint: print (unlike return) does not cause a function to exit.

```
python3 -m pytest -k if_statements --unlock
```

```
>>> def ab(c, d):
...     if c > 5:
...         print(c)
...     elif c > 7:
...         print(d)
...     print('foo')
>>> ab(10, 20)
```

```
_____  
_____  
>>> def bake(cake, make):
...     if cake == 0:
...         cake = cake + 1
...         print(cake)
...     if cake == 1:
...         print(make)
...     else:
...         return cake
...     return make
>>> bake(0, 5)
```

```
_____  
_____  
_____  
>>> bake(1, "yum")
```

```
_____  
_____  
>>> bake(2, 3)
_____
```

Defining Functions

Q3: Pick a Digit

Implement pick_digit, which takes positive integer n and non-negative integer k and has only a single return statement as its body. It returns the digit of n that is k positions to the left of the rightmost digit (the one's digit). If k is 0, return the rightmost digit. If there is no digit of n that is k positions to the left of the rightmost digit, return 0.

Hint: Use // and % and the built-in pow function to isolate a particular digit of n.

```
def pick_digit(n, k):
    """Return the k-th digit from the right of n.
```

```
>>> pick_digit(3579, 2)
5
>>> pick_digit(3579, 0)
9
>>> pick_digit(3579, 10)
0
"""
return _____
```

```
python3 -m pytest -k pick_digit
```

Q4: Middle Number

Implement middle by writing a single return expression that evaluates to the value that is neither the largest or smallest among three different integers a, b, and c.

Hint: Try combining all the numbers and then taking away the ones you don't want to return by using the built-in min and max functions:

```
>>> max(1, 2, 3)
```

```

3
>>> min(-1, -2, -3)
-3

def middle(a, b, c):
    """Return the number among a, b, and c that is not the smallest or largest.
    Assume a, b, and c are all different numbers.

    >>> middle(3, 5, 4)
    4
    >>> middle(30, 5, 4)
    5
    >>> middle(3, 5, 40)
    5
    >>> middle(30, 5, 40)
    30
    """
    return ____

python3 -m pytest -k middle

```

Iteration Using If and While

Q5: Falling Factorial

Let's write a function `falling`, which is a "falling" factorial that takes two arguments, `n` and `k`, and returns the product of `k` consecutive numbers, starting from `n` and working downwards. When `k` is 0, the function should return 1.

```

def falling(n, k):
    """Compute the falling factorial of n to depth k.

    >>> falling(6, 3) # 6 * 5 * 4
    120
    >>> falling(4, 3) # 4 * 3 * 2
    24
    >>> falling(4, 1) # 4
    4
    >>> falling(4, 0)
    1
    """
    """ YOUR CODE HERE """

python3 -m pytest -k falling

```

Q6: Divisible By k

Write a function `divisible_by_k` that takes positive integers `n` and `k`. It prints all positive integers less than or equal to `n` that are divisible by `k` from smallest to largest. Then, it returns how many numbers were printed.

```

def divisible_by_k(n, k):
    """Print all positive integers up to n that are divisible by k from smallest
    to largest, then return how many numbers were printed.

    >>> a = divisible_by_k(10, 2) # 2, 4, 6, 8, and 10 are divisible by 2
    2
    4
    6
    8
    10
    >>> a
    5
    >>> b = divisible_by_k(3, 1) # 1, 2, and 3 are divisible by 1
    1
    2
    3
    >>> b
    3
    >>> c = divisible_by_k(6, 7) # There are no integers up to 6 that are divisible by 7
    >>> c
    0
    """
    """ YOUR CODE HERE """

```

```
python3 -m pytest -k divisible_by_k
```

Q7: Double Eights

Write a function that takes in a number and determines if the digits contain two adjacent 8s.

```
def double_eights(n):
    """Return true if n has two eights in a row.
    >>> double_eights(8)
    False
    >>> double_eights(88)
    True
    >>> double_eights(2882)
    True
    >>> double_eights(880088)
    True
    >>> double_eights(12345)
    False
    >>> double_eights(80808080)
    False
    """
    """ YOUR CODE HERE """
```

```
python3 -m pytest -k double_eights
```

Syllabus Quiz

Q8: Syllabus Quiz

Please fill out the Syllabus Quiz, which confirms your understanding of the policies on the syllabus page.

Submit Assignment

Submit this assignment by running Provenance: Prepare Submission Bundle from the VS Code command palette and uploading the resulting zip to Gradescope. The zip already contains the files you've edited. Lab 00 has detailed instructions.

Your responses to WWPDP questions are not submitted, and they do not need to be. Lab credit is based on the code writing questions.

Optional Questions

These questions are optional. If you don't complete them, you will still receive credit for this assignment. They are great practice, so do them anyway!

Q9: Sum Digits

Write a function that takes in a nonnegative integer and sums its digits. (Using floor division and modulo might be helpful here!)

```
def sum_digits(y):
    """Sum all the digits of y.

    >>> sum_digits(10) # 1 + 0 = 1
    1
    >>> sum_digits(4224) # 4 + 2 + 2 + 4 = 12
    12
    >>> sum_digits(1234567890)
    45
    >>> a = sum_digits(123) # make sure that you are using return rather than print
    >>> a
    6
    """
    """ YOUR CODE HERE """
```

```
python3 -m pytest -k sum_digits
```